VIBE CODING

SOFTWARE DEVELOPMENT STANDARDS

BLUEPRINT

Control model, audit framework and rollout approach for AI-assisted software development

1 **Reference Model and Controls**
AI-assisted development reference model, role boundaries, measured risk asymmetry, maturity levels, control gates, scenarios, anti-pattern list and context discipline. p. 2 – 11

2 **Audit Framework and Rollout**
Category weights, scoring and pass decision, screening questions, decision log, glossary, tool mapping template, roadmap and the limits of the audit. p. 12 – 21

3 **Company Proposal**
Current-state summary, target control set and rollout plan for a specific organisation. Filled in with assessment findings. p. 22 – 25

AI-Assisted Software Development — Reference Model

Komtaş Software Standard

From intent to operations, the control points AI changes

A tool- and model-independent reference model. It defines which control is performed at which stage and by whom; it does not define which AI tool is used.

1. Intent and Context

- Business need and acceptance criteria
- Context package — code, schema, constraints
- AI rules files — versioned, reviewed
- Maturity level — prototype · pilot · production
- Data sensitivity class

2. Generation

- Code generation with AI assistant
- Prompt and context discipline
- Dependency selection — registry verification
- Reading the risk surface — authz, data, external calls
- Secrets and credentials — never inside the code

3. Verification

- Unit and integration tests
- Lint and type checking
- Local security scan
- Self-review — AI output is not assumed correct
- Check against acceptance criteria

4. Review

- Human review — mandatory
- Diff size limit — an unreadable diff is an unreviewed diff
- Attributability — who, when, what
- Declaration of the AI share
- Four eyes — where a team exists

5. Gate and Audit

- Secrets scanning
- SAST — rule set
- SCA and CVE scanning
- SBOM generation
- LLM triage — authorization, business logic, architecture
- Score and category grade
- Blocking finding gate

6. Deployment

- CI/CD pipeline
- Environment separation
- Versioning and release notes
- Rollback plan

7. Operations

- Monitoring and alerting
- Evidence archive — 90 days
- MTTR and trend
- Regression audit

Cross-Cutting Capabilities — apply at every stage

Rule Set Ownership and Governance

- Single source: SAT / Solution Architecture
- Threshold and detector acceptance rule
- Distribution through plugin versions

Security and Secrets Management

- Secret store and rotation
- Secure default configuration
- Access and authorization patterns

Traceability and Attributability

- Git record and commit discipline
- Audit trail: tool, rule, file, line
- Rationale for the change

Test Integrity

- Test presence and ratio
- Detection of deleted tests
- Coverage of acceptance criteria

Compliance and Evidence Production

- Framework view: ISO, SOC 2, ASVS, SSDF
- Three-state scale: evidence provided / partial / not produced
- Evidence package time series

Measurement and Thresholds

- Category weights sum to 100
- Material sufficiency threshold
- False positive rate measurement

NOTES

- The model is tool-independent; it does not determine which AI assistant is used.
- The stages are sequential, but not every stage is applied at the same depth in every project.
- The maturity level determines which controls are mandatory.

COLOUR LEGEND — CONTROL TYPE

- Human Decision
- Tool Capability
- Process Gate
- Governance

HOW TO USE THIS MODEL

This is a reference catalogue of control points. In a project standard, only the controls required by the maturity level are mandatory.

Role Boundaries — Who Does What, and What They Do Not Do

An AI assistant does not add a member to the team; it shifts the boundaries of existing roles. If the shifted boundaries are not written down, accountability falls into the gap.

ROLE	TYPE	DOES	DOES NOT
Developer	Human	Defines the intent and acceptance criteria · keeps the change to a reviewable size · reads the risk surface · owns the decision	Does not approve a change too large to be read · does not treat AI output as verified
AI Coding Assistant	Tool	Produces draft code, tests and documentation · suggests alternative approaches · speeds up repetitive work	Does not decide · carries no accountability · does not guarantee the correctness of what it produces
Human Review	Process	Examines the rationale and scope of the change · looks for business logic and authorization defects	Is not reduced to a style debate · does not repeat work the automated tools already do
Deterministic Audit Tools	Tool	Produce secrets, SAST, SCA, SBOM and configuration findings · repeatable and evidenceable	Do not interpret · cannot see authorization and business logic defects · produce false positives
LLM Triage	Human + Tool	Filters out false positives · looks for the authorization, business logic and architecture defects tools cannot see	Cannot close a blocking finding on its own · dismissal authority is limited per category
CI Gate	Process	Applies the thresholds · stops the merge when a blocking finding exists · produces the audit trail	Does not negotiate the threshold · is not made bypassable
Rule Set Owner (SAT)	Governance	Is the single source of the rule set, the thresholds and the detector acceptance rules	Does not define per-project exceptions · does not allow local copies to be edited
Product / Project Owner	Human	Determines the maturity level and data sensitivity · accepts or rejects the risk	Cannot remove risk by lowering a technical threshold
Internal Audit	Governance	Tracks the evidence package, the trend and open blocking findings	Does not make a compliance decision based on a single report

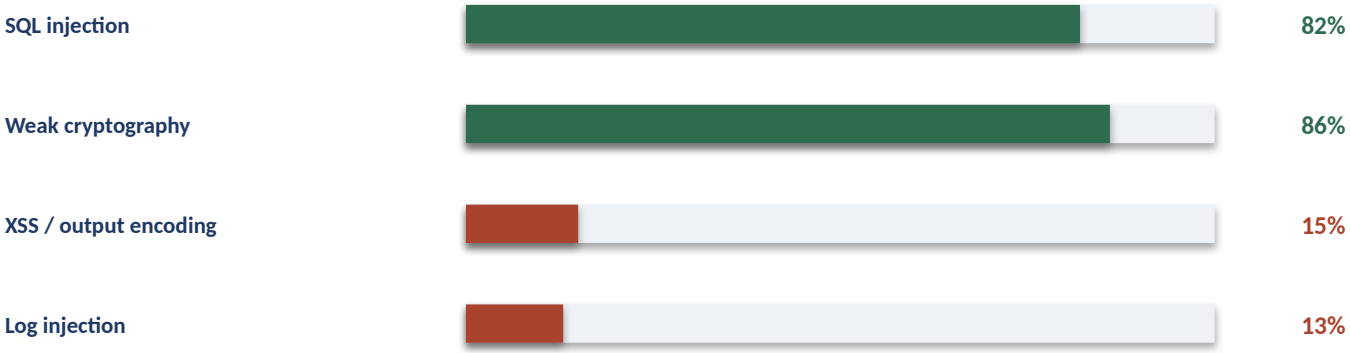
The owner of the decision is always a human. What is expected of the human is not reading every line; it is keeping the change to a readable size and reading the risk surface. A change too large to be reviewed is an unreviewed change.

The Measured Asymmetry of AI-Generated Code

AI-generated code does not perform equally across security categories. Control weight belongs where the model is weak.

SECURITY TEST PASS RATE · AI-generated code

Veracode measurement, March 2026 — pass rate by category



Models have learned the common and well-documented vulnerabilities. They are weak in sparsely sampled, context-dependent and "silent" categories.

WHAT THIS MEANS

- There is no answer to the general question "is AI code secure"; the question is asked per category.
- Control weight shifts to the categories where the model is weak: output encoding, log sanitization, CORS, SSRF, CSRF.
- Developer experience does not close this asymmetry; the weak categories are the hardest ones to spot by eye.
- If the SAST layer did not run, the statement "no findings in these categories" is void — nothing was scanned.

AI-SPECIFIC BLIND SPOTS — no equivalent in classical frameworks

Hallucinated package — M01

The AI suggests a package that does not exist; an attacker registers that name in the registry.

Slopsquatting — M02

A newly published, similarly named malicious package. A minimum package age check is required.

Rules file persistence — M18

If AI rules files are not versioned and reviewed, they produce silent behaviour changes.

Hidden / bidi Unicode — M19

Invisible characters separate the code as read from the code as executed.

Business logic and authorization — M23

Defects the deterministic tools cannot see; a separate review layer is required.

Loss of commit discipline — M17

Large, unexplained commits make attributability and review impossible.

Three Maturity Levels — Control Scales with the Level

Not all code goes through the same controls. The level is set by who and what data the code touches, not by the development method.

1. PROTOTYPE

Produced to see whether an idea works. It has no users and its data is synthetic.

MANDATORY CONTROLS

- Secrets scan (pre-commit)
- Hidden / bidi Unicode check
- Package name verification

RELAXABLE AT THIS LEVEL

- SAST threshold
- Test coverage
- SBOM generation
- Mandatory human review

EXIT CONDITION — It cannot remain at prototype level once it will touch real users or real data.

2. PILOT

Opened to a limited and known user group. Real data may enter; rollback is possible.

MANDATORY CONTROLS

- All prototype controls
- SAST and SCA scanning (advisory)
- SBOM generation and retention
- Human review — at least one approval
- Audit trail: who deployed what and when

RELAXABLE AT THIS LEVEL

- Score threshold (not blocking)
- Coverage threshold
- Framework mapping reporting

EXIT CONDITION — Moving to production is not advised until the score stays stably above 85 for 4–6 weeks.

3. PRODUCTION

Opened to general use or to a regulated customer. Rollback is costly.

MANDATORY CONTROLS

- All pilot controls
- CI gate in blocking mode
- Blocking finding = no merge
- LLM triage layer
- Evidence package — dated archive
- Rule set from the centre, no local copy

RELAXABLE AT THIS LEVEL

- None — control is not relaxed at this level; scope is narrowed

EXIT CONDITION — A need to relax control is resolved by reducing scope, not by lowering the level.

Data sensitivity raises the level; it never lowers it. A pilot processing personal data is subject to production controls; no component opened to a regulated customer is left at pilot level.

Control Gates — Where to Stop, Who Decides

The gates are applied in order and each catches a different class of defect. Skipping one gate does not mean a later gate will catch that class.

	GATE	WHAT IS VERIFIED	DECISION	IF IT DOES NOT PASS
G1	**Intent and acceptance criteria**	Is it written down what will be built and what counts as “done”; is the scope boundary defined	Developer	Generation does not start — undefined intent produces unverifiable output
G2	**Pre-commit**	Secrets, hidden / bidi Unicode, large binary files	Automated	No commit is created — once a secret enters history, it needs rotation, not deletion
G3	**Dependency verification**	Does the package really exist, how old is it, what is its licence, are there known vulnerabilities	Automated	The package is not added — hallucinated packages and slopsquatting are stopped at this gate
G4	**Static analysis**	SAST, configuration, output encoding, log sanitization, CORS, SSRF	Automated	The finding is written to the PR; a warning in advisory mode, a block in blocking mode
G5	**Human review**	Is the size of the change reviewable; what is the authorization and business logic impact	Human	No merge — an unreadable diff is split first; this is the only layer the tools cannot cover
G6	**LLM triage**	Dismissal of false positives and the search for defects the tools cannot see	Human + Tool	Dismissal without a rationale is not accepted; it cannot close a blocking finding on its own
G7	**Score and threshold**	Category-weighted score; is there a blocking finding	Automated	If there is a blocking finding, it does not pass regardless of the score
G8	**Evidence archive**	Is the finding set stored with a date; can the trend and open findings be tracked	Governance	A single report is not compliance evidence; improvement cannot be shown without a time series

Human decision

Automated / process gate

Governance

G2–G4 and G7 are automated; G1 and G5 cannot be delegated. G6 and G8 depend on the rule set owner.

Scenario — A Feature Built End to End with an AI Assistant

The assistant enters at steps 4–6. The steps before and after are the same whether or not an assistant is used.

STEP	STAGE	WORK PERFORMED	OUTPUT
INTENT AND CONTEXT · before generation			
1	Intent	The need, the acceptance criteria and what is out of scope are written down	Defined work item
2	Intent	The maturity level and data sensitivity are determined	Control set to be applied
3	Context	The context given to the assistant is selected: relevant modules, contracts, rules file	Limited and purposeful context
GENERATION · code and tests			
4	Generation	The assistant produces a code draft; the developer compares the alternatives	Code draft
5	Generation	The developer splits the change into reviewable parts and reads the risk surface	Owned change
6	Generation	Tests are written in a step independent of the assistant and derived from the acceptance criteria	Independent test set
VERIFICATION · automated layer			
7	Verification	Pre-commit: secrets, hidden / bidi Unicode, large binary files	Clean commit
8	Verification	Dependencies are verified: existence, age, licence, known vulnerabilities	Verified dependency list
9	Verification	SAST, configuration and AI blind spot rules run	Finding set
10	Verification	An SBOM is produced and stored together with the commit	Versioned SBOM
REVIEW · human and LLM			
11	Review	Human review: rationale, scope, authorization and business logic impact	Review approval
12	Review	LLM triage: dismissal of false positives and the search for defects the tools cannot see	Triage result with rationale
GATE · decision			
13	Gate	Score and blocking finding check; decision in advisory or blocking mode	Gate decision
DEPLOYMENT			
14	Deployment	Who deployed what, with which commit — written to the audit trail	Deployment record
OPERATIONS			
15	Operations	Runtime monitoring; the finding set is added to the archive with a date	Evidence package entry

Step 6 is written separately because a test coming from the same generation as the code means the code verifies its own claim. Tests are derived from the acceptance criteria, not from the generated code.

Scenario — A Suggested Dependency Is Verified

AI assistants can produce package names that do not exist. The attacker's method is to register those frequently produced names in the registry in advance.

STEP	TYPE	CHECK	RESULT
SUGGESTION · assistant output			
1	**Suggestion**	The assistant suggests a package for a function and writes the import line	Candidate dependency
VERIFICATION QUESTIONS · in order, the first failure stops the chain			
2	**Question**	Does the package really exist in the registry	If not: hallucinated package — rejected
3	**Question**	How many days ago was the package published	Below the minimum age: put on hold
4	**Question**	Is the name one character away from a common package	Similar name: slopsquatting suspicion — manual approval
5	**Question**	Are download counts, version history and maintenance signals consistent	Single version, no history: rejected
6	**Question**	Is the licence on the organisation's deny list	Deny list: rejected
7	**Question**	Are there known vulnerability records	Critical vulnerability: version pinned or rejected
CALIBRATION			
8	**Exclusion**	Monorepo local packages are excluded from the registry query	Exclusions are written into the report
RECORD			
9	**Record**	The accepted dependency is written into the SBOM	Versioned SBOM
10	**Record**	The rejection or acceptance decision is written to the decision log with its rationale	Decision record

Step 8 is a calibration item and comes from a measured error: querying monorepo local packages against the registry produced 52 false blocking findings in a single run. Exclusions are not deleted from the report; they are written as an information line.

Scenario — From Finding to Closure

The order holds: the tool produces, the LLM interprets, the decision is made by deterministic thresholds. Breaking the order means interpretation replaces measurement.

STEP	STAGE	WORK PERFORMED	OUTPUT
GENERATION · deterministic layer			
1	Finding	The deterministic tool layer produces the finding; source file, line and rule id are recorded	Raw finding
2	Context	The surrounding code and call context are added to the triage input	Finding with context
DISMISSAL · false positives			
3	Dismissal	Is it a false positive — a rationale is written; dismissal without one is not accepted	Dismissed or confirmed
4	Boundary	In categories where dismissal authority is closed, the finding cannot be dismissed	Mandatory review
EXTENSION · what the tools cannot see			
5	Search	Areas the tools cannot see are searched separately: authorization, business logic, architecture	LLM-sourced finding
6	Authority	An LLM-sourced finding is added to the score; it cannot close the gate on its own	Asymmetric authority applied
CLASSIFICATION AND DECISION			
7	Classification	Context does not silence a finding, it reclassifies it — the rationale stays in the report	Applicable / not applicable
8	Decision	The score is recalculated; if a blocking finding exists, the score is disregarded	Gate decision
CLOSURE AND EVIDENCE			
9	Closure	A fix is applied or the risk is accepted with a rationale; both are written down	Closure record
10	Archive	The finding set is added to the archive with date and commit; its lifetime is tracked	Evidence package

The dismissal rate is written openly into the report. If how many findings triage dismissed is hidden, the report's reliability becomes unmeasurable; a high dismissal rate shows that either the rule set or the dismissal needs calibration.

What Not to Do

Common mistakes in AI-assisted development; why they look attractive, what they break and what replaces them.

WHAT NOT TO DO	WHY IT LOOKS ATTRACTIVE	WHAT BREAKS	INSTEAD
Approving a large AI diff without reading it	Assistant code looks tidy and confident; as the change grows, reading it becomes impossible and approval turns into a formality	Authorization and business logic defects are caught by no tool; the only layer that could catch them is closed	**The change is split to a reviewable size; the risk surface is read and the rest is verified by behaviour**
Having the same assistant write the tests too	Writing tests is costly; produced in the same session it looks free	The code verifies its own claim; the gap between declaration and reality stays invisible	**Tests are derived from the acceptance criteria and written in a separate step**
Giving the assistant the whole repository as context	More context is assumed to give better answers	Secrets and personal data leave unnecessarily; answer quality drops with context noise	**Context is selected on purpose: relevant modules, interface contracts and the rules file**
Not versioning the rules files	They look like a local settings file, not worth reviewing	The assistant's behaviour changes silently; different developers produce different standards in the same repository	**AI rules files are part of the code: versioned, reviewed, distributed from the centre**
Making large, unexplained commits	The assistant changes many files together; splitting them feels like effort	Review becomes impossible in practice and attributability is lost; why a change was made cannot be reconstructed afterwards	**A commit carries a single intent and states its rationale**
Opening the gate in blocking mode on day one	The aim is for the control to be taken seriously; strictness feels reassuring	The team finds a way to bypass the audit; the control stays on paper	**Run advisory for 4–6 weeks; switch to blocking once the score is stably above 85**
Treating the score as the pass decision	A single number is easy to explain to management	A blocking finding hides behind a high score	**If there is a blocking finding it does not pass regardless of the score; the score only tracks improvement**
Treating a small code base's high score as success	A repository that trips no rule looks clean	A score produced with no material to measure comes out higher than one measuring a real repository	**A code base below the threshold is not scored; the result is written as "could not be assessed"**

Every item on this list speeds things up in the short term. The cost appears at the first audit, the first security incident, or the first time the code is handed to someone else.

Prompt and Context Discipline

What is given to the assistant determines both the quality of the output and the organisation's data boundary. Context selection is a policy decision, not a technical one.

GIVEN

- The module being changed and its direct dependencies
- Interface contracts, type definitions, schema files
- Acceptance criteria and what is out of scope
- The project's rules file — in its versioned and reviewed form
- An example of how a similar task was solved before
- The related defect record or test output, if any

NOT GIVEN

- Production secrets, keys, tokens and connection strings
- Sample records containing real personal data
- Customer contracts or documents that constitute trade secrets
- Unrelated modules — context noise lowers answer quality
- The whole repository; broad context "just in case"
- Code or configuration belonging to another customer

WRITTEN DOWN

- Which assistant and which version were used
- What context was given — at the level of a file list
- The state of the rules file at that commit
- Which part of the generated code was accepted and which was rewritten
- The rejected alternative and the reason for rejection
- The developer who owns the decision

FREQUENTLY ASKED

What is a rules file

Configuration living inside the repository that steers the assistant's behaviour. It is part of the code: versioned and reviewed.

Why distributed centrally

If local copies are edited, different standards form in the same repository; the audit means something because everyone is measured by one rule set.

Why written down

When a finding arrives six months later, root cause analysis is impossible if the context the code was produced with cannot be reconstructed.

Where is the boundary

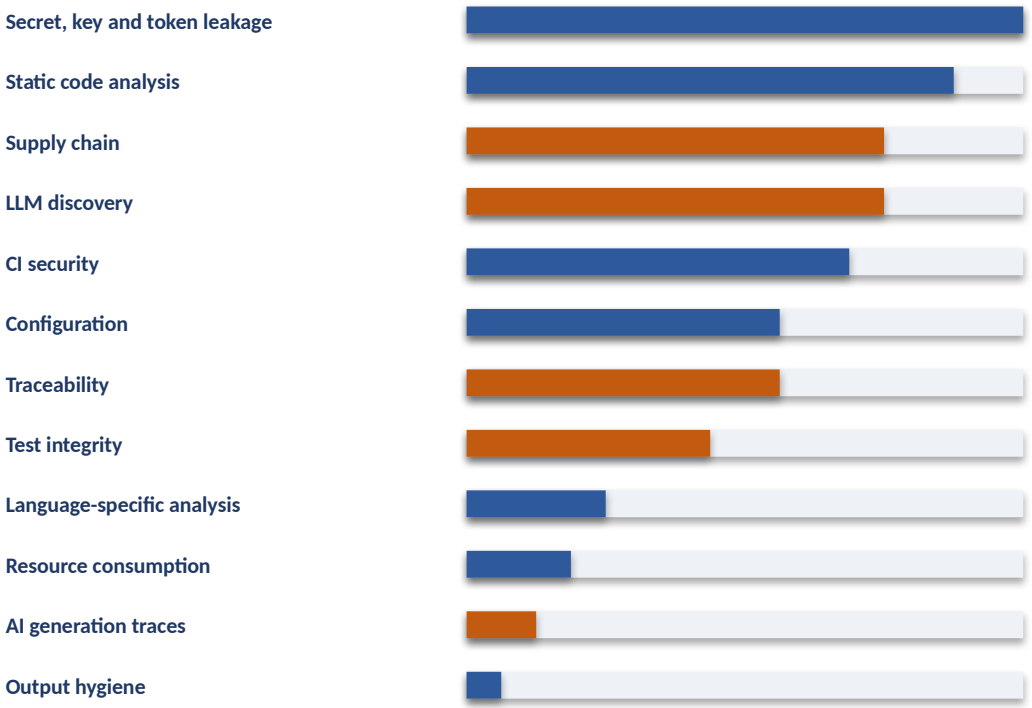
Everything given to the assistant counts as having left the organisation; contracts and data classification are applied on that assumption.

The context given to the assistant is subject to the organisation's data classification policy. Where that policy is not written, context discipline cannot be applied either.

Audit Framework — 23 Items, 12 Categories, Weighted Score

The weights sum to 100 and do not change with the code base. The distribution follows the measured weak categories of AI-generated code.

CATEGORY WEIGHTS



Secret, key and token leakage	16	Credentials in code, configuration and committed history
Static code analysis	14	Injection, output encoding, access and input validation defects
Supply chain	12	Package existence, age, licence, known vulnerabilities, SBOM integrity
LLM discovery	12	Authorization, business logic and architecture defects tools cannot see
CI security	11	Permissions, secrets management, pipeline integrity and gate configuration
Configuration	9	Certificate validation, defaults, open endpoints and weak settings
Traceability	9	Commit discipline, attributability, audit trail and change rationale
Test integrity	7	Test independence, coverage, and whether the claim matches reality
Language-specific analysis	4	Common defect patterns originating in Python
Resource consumption	3	Unbounded loops, uncontrolled memory and cost leakage
AI generation traces	2	Hidden / bidi Unicode, rules file persistence, generation residue
Output hygiene	1	Cleanliness of produced artefacts and report outputs

Categories carrying risk specific to AI-generated code — total weight 42

Classical security categories — total weight 58

Why the weight sits here

Measurement shows AI-generated code is strong in injection and cryptography and weak in silent categories such as output encoding and log sanitization. The centre of weight has been shifted to that weak side.

Who changes the weights

Weights and thresholds are managed centrally; no per-project exception is defined. If a threshold is believed to be wrong, the local copy is not edited — the rule set owner is petitioned with a rationale.

How a new detector is admitted

Four conditions: a vulnerable and a correct implementation fixture, a declared false positive rate, an information → warning → block rollout order, and, if a new category is opened, the rationale for which category the weight is taken from.

Scoring and the Pass Decision — They Are Not the Same Thing

The score is there to track improvement. The pass decision is determined by the presence of a blocking finding; a high score does not close one.

LETTER GRADE

A	90 and above
B	80 – 89
C	70 – 79
D	60 – 69
F	below 60

The score only tracks the improvement trend.

PASS DECISION

- If there is a blocking finding the project does not pass — whatever the score.
- An LLM-sourced finding is added to the score; it cannot close the gate on its own.
- A high-confidence LLM finding is weighted like a blocking one.
- If the SAST layer did not run, the result is not valid; the scope is incomplete.

MATERIAL SUFFICIENCY

If the code base is smaller than 5 source files or 200 lines of code, no score is produced; the result is “COULD NOT BE ASSESSED” and no framework control receives “Evidence provided”.

A measured example: a one-line “hello world” scored 89/100 under naive scoring while a real enterprise repository scored 45. Tripping no rule is not the same as being clean.

THE ORDER HOLDS — THREE LAYERS AND THEIR AUTHORITY

Deterministic tool layer — Measurement

Produces the finding. Repeatable, evidenceable, without interpretation.

→

LLM layer — Interpretation

Interprets and dismisses findings and searches for what the tools cannot see.

→

Threshold layer — Decision

Makes the decision. Deterministic, closed to negotiation.

Breaking the order — the LLM deciding on its own finding — reduces the audit from measurement to interpretation. Asymmetric authority exists for this reason: the LLM cannot open the gate, it can only ask for it to be closed.

RULES APPLIED TO THE REPORT ITSELF

The false positive rate is written into the report

How many findings triage dismissed is a reliability indicator; it is not hidden.

Out-of-scope areas are written into the report

Load testing, DAST, cross-file taint and licence provenance are outside this audit.

A file that cannot be parsed is a file that was not scanned

It stays blocking and is reported by file name; it is not silently skipped.

A category with no detector produces a warning

The absence of findings does not mean the scan was clean — nothing was scanned.

Screening Questions — Usage, Generation and Verification

Questions that establish the current state and which controls are missing. The maturity level and the roadmap follow from these answers.

#	QUESTION	WHAT IT DETERMINES	RESULT BY ANSWER
1	**In which teams and for which work is the AI coding assistant used**	**The scope**	If usage is widespread the standard becomes mandatory; if scattered, an inventory comes first
2	**Which assistant and which deployment model — cloud service or on-premises**	**The data boundary**	Cloud service: context leaves the organisation; a data classification policy becomes a precondition
3	**Is the boundary of the context given to the assistant written down**	**Policy maturity**	If not written, the context policy is written first; a technical control alone is not enough
4	**Are the rules files versioned and reviewed**	**Behavioural consistency**	If not versioned, different standards have formed in the same repository; this is closed first
5	**Who writes the tests for the generated code, and how**	**Verification independence**	If the tests also come from the assistant there is no verification layer; an independent test step is established
6	**Is human review mandatory or optional**	**The presence of the non-delegable layer**	If optional, authorization and business logic defects are being caught at no layer at all
7	**Is there a pre-commit check — secrets, hidden Unicode, large files**	**The presence of the first gate**	If absent, the cheapest gate is missing; this is the first control to establish
8	**How are dependencies verified — package existence, age, licence**	**Supply chain risk**	Without verification, hallucinated packages and slopsquatting are open
9	**Is an SBOM produced and retained**	**Whether evidence can be produced**	If not produced, impact analysis is impossible when a vulnerability advisory arrives
10	**Which categories does SAST cover — are output encoding and log sanitization included**	**The reality of the scope**	If not included, the weakest categories of AI code are not being scanned at all
11	**What is the commit discipline — is there a size and explanation expectation**	**Attributability**	With large, unexplained commits, root cause analysis cannot be reconstructed afterwards

Every unanswered question is an assumption. Assumptions are written explicitly into the proposal and moved to the list of items to be clarified before contract.

Screening Questions — Gate, Governance and Scope

Questions that establish whether the control is binding and whether evidence can be produced.

#	QUESTION	WHAT IT DETERMINES	RESULT BY ANSWER
12	**Is there a CI gate; does it run in advisory or blocking mode**	**How binding the control is**	Without a gate, findings are produced but not consumed; advisory is established first
13	**Is the definition of a blocking finding written down**	**Decision clarity**	If not written, the gate decision varies by person; the threshold is defined centrally
14	**Does the rule set come from the centre, or can local copies be edited**	**Comparability of the measurement**	If local copies exist, projects are not being measured by the same yardstick
15	**Who holds the authority to change thresholds**	**Governance clarity**	If it sits with the project team the threshold erodes over time; authority is held by a single owner
16	**Is the finding set retained with dates**	**Evidence continuity**	If not retained, improvement cannot be shown; an enterprise customer asks for a time series
17	**Is the average closure time of open blocking findings measured**	**Process health**	If not measured, findings accumulate; closure time alone is a maturity indicator
18	**Is triage performed; are dismissal rationales recorded**	**The reliability of the report**	If dismissals have no rationale, the report is an opinion rather than a measurement
19	**What is the team size and number of authors**	**Which controls are applicable**	In a single-author project a four-eyes finding is classified “not applicable”, not deleted
20	**Which deployment target does the code serve — internal tool, product, regulated customer**	**The maturity level**	Regulated customer: production controls are mandatory and the gate becomes blocking
21	**What is the data sensitivity — is there personal or financial data**	**The control set**	With personal data the level is raised; it is not left at pilot level
22	**Which compliance frameworks are expected to receive evidence**	**The report scope**	Without an expectation, framework mapping is noise; with one, an evidence status table is added to the report

These questions are not an audit; they establish the scope in which an audit is meaningful. The answers are written into the report, because when a result changes between two audits it must be possible to tell whether that came from the code or from the context.

Decision Log — Usage, Accountability and Control Architecture

For each decision: the rationale, the rejected alternative, and under which condition it is revisited.

#	TOPIC	DECISION TAKEN	RATIONALE AND ALTERNATIVE	WHAT WOULD CHANGE IT
1	**Assistant usage**	**Permitted — not banned, brought under control**	A ban is unenforceable; it produces shadow usage and removes visibility entirely. Rejected: permission for some teams, a ban for others.	If a contract or regulation bans external service use on a given project, that project is taken out of scope
2	**Accountability**	**The owner of the decision is a human; assistant output is not evidence**	Delegating accountability to a tool means no defect has an owner. Rejected: treating “the assistant produced it” as mitigating.	Does not change — this decision has no dependent condition
3	**Test independence**	**Tests are derived from acceptance criteria and written in a separate step**	A test from the same generation means the code verifies its own claim. Rejected: producing code and tests in the same session.	If an independent verification layer such as mutation testing is established, the steps may be merged
4	**Control order**	**The tool produces, the LLM interprets, the threshold decides**	Interpretation replacing measurement removes repeatability from the audit. Rejected: the LLM issuing a pass/fail decision directly.	Does not change — if the order breaks, the result is not an audit
5	**LLM authority**	**Asymmetric — it can add findings, it cannot close the gate**	The cost of a false positive and the cost of a missed finding are not symmetric. Rejected: full dismissal authority for the LLM.	High-confidence LLM findings are weighted like blocking ones; this setting can be calibrated
6	**Rule set ownership**	**A single source from the centre; local copies are not edited**	The audit means something because everyone is measured by the same rule set. Rejected: per-project threshold exceptions.	A threshold change is petitioned to the rule set owner with a rationale; if accepted it is released to everyone
7	**Material threshold**	**No score is produced below 5 files or 200 lines**	A small code base trips no rule and scores higher than a real project. Rejected: giving every code base a score.	The thresholds can be calibrated; they cannot be removed

Decision Log — Rollout, Governance and Reporting

For each decision: the rationale, the rejected alternative, and under which condition it is revisited.

#	TOPIC	DECISION TAKEN	RATIONALE AND ALTERNATIVE	WHAT WOULD CHANGE IT
8	**Rollout**	**Advisory for the first 4–6 weeks, then blocking**	Early hardening leads the team to bypass the audit and leaves the control on paper. Rejected: blocking from day one.	Blocking is not enabled until the score is stably above 85
9	**Pilot scope**	**Start with three repositories**	A narrow pilot makes calibration errors visible at low cost. Rejected: rolling out to all repositories at once.	Scope widens once the false positive rate in the pilot falls to an acceptable level
10	**Use of context**	**A finding is not silenced, it is reclassified**	Otherwise context information turns into a way of escaping the audit. Rejected: removing non-applicable findings from the report.	Does not change — loss of scope stays visible in every case
11	**Compliance language**	**Three-state: evidence provided / partial evidence / evidence not produced**	A code audit cannot make a compliance decision; that is made by an accredited body, a CPA or a QSA. Rejected: the phrase “you are not compliant”.	Does not change — this is a limit of authority, not a preference
12	**Framework scope**	**AI-specific items are kept outside the framework mapping**	Forcing items with no equivalent in classical frameworks lowers the report’s credibility. Rejected: assigning a control number to every item.	If the relevant frameworks define controls for AI code, the mapping is reopened
13	**Evidence retention**	**The finding set is archived with date and commit**	An enterprise customer asks for a time series, not a single report; improvement can only be shown that way. Rejected: retaining only the latest report.	Retention period and location follow the organisation’s data retention policy
14	**Scope declaration**	**Out-of-scope areas are written in every report**	Load testing, DAST, cross-file taint and licence provenance are outside this audit. Rejected: presenting a “pass” result as general assurance.	As scope widens this list shortens; it is never claimed to be empty

Glossary

Shared definitions of the concepts used in proposals and presentations. Used to separate a debate about terms from a debate about controls.

Vibe coding
A way of working in which code is largely produced by an AI assistant while the developer defines the intent and steers the output.

AI coding assistant
A model-based tool that produces draft code, tests and documentation. It does not decide and carries no accountability.

Rules file
Configuration living inside the repository that steers the assistant's behaviour. It is part of the code: versioned and reviewed.

Hallucinated package
The model producing a package name that does not exist. An attacker registers that name in the registry and enters the code.

Slopsquatting
A supply chain attack carried out by registering frequently produced wrong package names in the registry in advance.

Declaration debt
Code not actually applying the control it claims to apply. If the test comes from the same generation, it goes unnoticed.

Bidi / hidden Unicode
Characters that are invisible or change writing direction. They separate the code as read from the code as executed.

SAST
Static application security testing. Searching for defect patterns in the source without running the code.

SCA
Software composition analysis. Examining dependencies for version, licence and known vulnerabilities.

SBOM
Software bill of materials. A machine-readable inventory of which component at which version is used.

Output encoding
Escaping data according to the context it is going into. The primary defence against XSS.

Log sanitization
Cleaning input written to logs. Without it an attacker can manipulate the log record.

SSRF
Forcing the server to make a request to an address determined by the attacker.

CORS
The browser's cross-origin access rule. A loose configuration opens data to another domain.

Triage
The process of dismissing, prioritising and justifying findings.

False positive
A case the tool marks as a finding that is not actually a defect. The rate is written into the report.

Blocking finding
A finding whose presence alone prevents passing. It cannot be offset by the score.

Advisory / blocking mode
Whether the gate warns or stops the merge. Rollout starts with advisory.

Attributability
Being able to reconstruct afterwards who made a change and for what reason.

Evidence package
A time series of dated finding sets. Contains the trend, finding lifetime and closure time.

Tool Mapping Template — Tool Equivalents of the Control Blocks

The equivalent of each control block in common tool families. The choice is determined by the organisation's existing platform, skills and licensing.

Secrets scanning

Open source tool
detect-secrets, gitleaks

Development platform built-in
Push protection, secret scanning

Enterprise audit framework
Weight 16 · blocking · pre-commit

Static analysis (SAST)

Open source tool
semgrep, bandit

Development platform built-in
Code scanning, built-in rule sets

Enterprise audit framework
Weight 14 · AI blind spot rules in addition

Dependency analysis (SCA)

Open source tool
pip-audit, OSV query

Development platform built-in
Dependency alerts, automated upgrades

Enterprise audit framework
Weight 12 · package age and existence check

SBOM generation

Open source tool
CycloneDX, SPDX

Development platform built-in
In the build step via configuration

Enterprise audit framework
Stored together with the commit

Configuration audit

Open source tool
config-lint, policy as code

Development platform built-in
Environment and secrets management settings

Enterprise audit framework
Weight 9 · comment/code separation applied

CI gate

Open source tool
Script-based threshold check

Development platform built-in
Required checks, branch protection

Enterprise audit framework
Advisory → blocking transition rule

LLM triage

Open source tool
None — model access required

Development platform built-in
None

Enterprise audit framework
Weight 12 · asymmetric authority · rationale mandatory

Evidence archive

Open source tool
File-based dated archive

Development platform built-in
Artefact retention

Enterprise audit framework
Time series · MTTR · open finding tracking

Open source tool

Development platform built-in

Enterprise audit framework

A filled cell does not mean the tool is sufficient

Capability depth and coverage are assessed per block; a category with no detector is written into the report as a warning.

The LLM triage row stays empty

This layer has no open source or platform equivalent; it requires model access and rationale discipline.

Tool names change often

Product and feature naming in this area changes rapidly; current naming should be confirmed before a proposal.

Rollout Roadmap

Control is not hardened in a single step. Early hardening leads the team to bypass the audit and leaves the control on paper.

PHASE 0 · Preparation

1 – 2 weeks

- The rule set owner and threshold change authority are written down
- Maturity level and data sensitivity definitions are agreed
- Three repositories are selected for the pilot
- The context and rules file policy is written

EXIT CONDITION — The pilot does not start unless ownership and thresholds are written down

PHASE 1 · Advisory

4 – 6 weeks

- The gate is established in advisory mode; no merge is blocked
- The finding set is archived with a date on every run
- The false positive rate is measured and the rule set is calibrated
- The team is trained on reading findings and on triage

EXIT CONDITION — The next phase is not entered until the false positive rate falls to an acceptable level

PHASE 2 · Score monitoring

4 – 8 weeks

- The score trend is tracked; closure time for blocking findings is measured
- The triage process and dismissal rationale discipline take hold
- Scope is widened gradually beyond the pilot
- Framework mapping is enabled on projects that expect it

EXIT CONDITION — Blocking is not enabled until the score is stably above 85

PHASE 3 · Blocking

continuous

- The gate moves to blocking mode; a blocking finding stops the merge
- The evidence package is produced and retained regularly
- Gaps longer than 14 days without an audit are reported
- New detectors are rolled out in the order information → warning → block

EXIT CONDITION — There is no going back; a threshold reduction request goes to the rule set owner

The pilot is kept to three repositories. A narrow pilot makes calibration errors visible at low cost: in one measured run, monorepo local packages produced 52 false blocking findings.

No phase is skipped. Where blocking is enabled without an advisory stage, the control becomes a formality to be bypassed and loses its measurement value.

The Limits of the Audit and Compliance Framework Language

A code audit cannot make a compliance decision. Writing down what was not measured protects the value of what was.

OUT OF SCOPE — WRITTEN IN EVERY REPORT

Load and resilience testing	Performance and capacity behaviour are not measured in this audit.
DAST / running application testing	The audit is on source code; no request is sent to a running application.
Cross-file taint analysis	Data flow across files is not traced; defect patterns are sought at file level.
Licence provenance	The licence declaration is read; its accuracy is not separately verified.
Business process and organisation	Separation of duties and authorization processes cannot be measured from code.
Infrastructure and network security	Runtime environment, network segmentation and access controls are out of scope.

A “pass” result is not assurance for these areas.

THREE-STATE EVIDENCE SCALE

Evidence provided

Code-level evidence has been produced that the control operates effectively.

Partial evidence

Part of the evidence has been produced; the remainder is outside the scope of this audit.

Evidence not produced

The relevant evidence could not be produced in this repository. It does not mean the control does not operate.

Phrase not used: “you are not compliant”. This is a limit of authority, not a preference.

WHO MAKES THE COMPLIANCE DECISION

ISO 27001
Accredited certification body

SOC 2
Licensed CPA firm — issues an opinion; there is no pass/fail

PCI DSS
QSA

CRA
Manufacturer or notified body

Language used

“This finding weakens the evidence that the relevant control operates effectively.” / “The evidence to be presented to the auditor in this scope could not be produced in this repository.”

Why this way

The standards certify the management system rather than the code; some controls have no code-level measurable equivalent, and scope is set by a statement of applicability.

AI-specific items

Items with no equivalent in classical frameworks are kept outside the mapping. Not forcing them raises the report’s credibility.

SECTION 3

COMPANY PROPOSAL

The preceding pages are organisation-independent and can be used both as an internal standard and as a proposal annex. The pages in this section are filled in with assessment findings and belong to one organisation only.

Current-state summary

The answers to the screening questions, the missing controls and the risk each corresponds to.

Target control set

Maturity level, gates to be applied, thresholds and what is left out of scope.

Rollout plan

Phase schedule, pilot scope, responsibilities and open items.

Company Proposal — Current-State Summary

The answers to the screening questions, the missing controls and the risk they correspond to. Filled in after the assessment.

#	ASSESSMENT AREA	CURRENT STATE	CORRESPONDING RISK	CRITICALITY
1	Usage spread and assistant selection			
2	Context and rules file policy			
3	Test independence			
4	Mandatory human review			
5	Pre-commit controls			
6	Dependency verification and SBOM			
7	Static analysis coverage			
8	CI gate and thresholds			
9	Rule set ownership			
10	Evidence archive and traceability			

Every missing control is tied to a corresponding risk; an item whose risk cannot be written does not enter the report. Items not applicable due to team structure are not deleted but marked “not applicable — rationale”.

Company Proposal — Target Control Set

Maturity level, gates to be applied, thresholds and what is left out of scope. Read together with the control gates page.

LEVEL 1

Prototype

selected / not selected — to be marked

LEVEL 2

Pilot

selected / not selected — to be marked

LEVEL 3

Generation

selected / not selected — to be marked

GATES TO BE APPLIED

Which of G1–G8, and in which mode — advisory or blocking

THRESHOLDS AND WEIGHTS

The rule set version in use and any justified deviations

LEFT OUT OF SCOPE

Which repositories, which categories, on what grounds

OPEN ITEMS AND ASSUMPTIONS

Unanswered screening questions and the assumptions to be written into the proposal

A threshold deviation is not defined per project. Every deviation written on this page must have been petitioned to and accepted by the rule set owner.

Company Proposal — Rollout Plan

Phase schedule, pilot scope, responsibilities and open items.

PHASE	SCOPE	START	END	OWNER AND EXIT CONDITION

PILOT SCOPE

The three repositories selected, the rationale and the expected finding profile

TOOL SELECTION AND RATIONALE

The tools selected per block and their fit with the existing platform

OPEN ITEMS

Items to be clarified and the responsible party

Every phase has an exit condition. A phase with no written exit condition extends indefinitely, and a control left in advisory mode turns over time into a report nobody reads.